Experiment: 05

Aim: Connect an Android application to the Internet.

Objective:

The primary objective is to learn how to establish network connections send HTTP Requests and handle responses in an Android application, enabling features such as fetching data from API or accessing web services.

Materials:

1. Android studio installed on your computer.
2. Basic understanding of Java programming and Android development.

Procedure:

1. Launch Android studio.
2. Create a new project and select template as empty activity.
3. Add Internet permission to Android manifest.
4. Implement network connection.

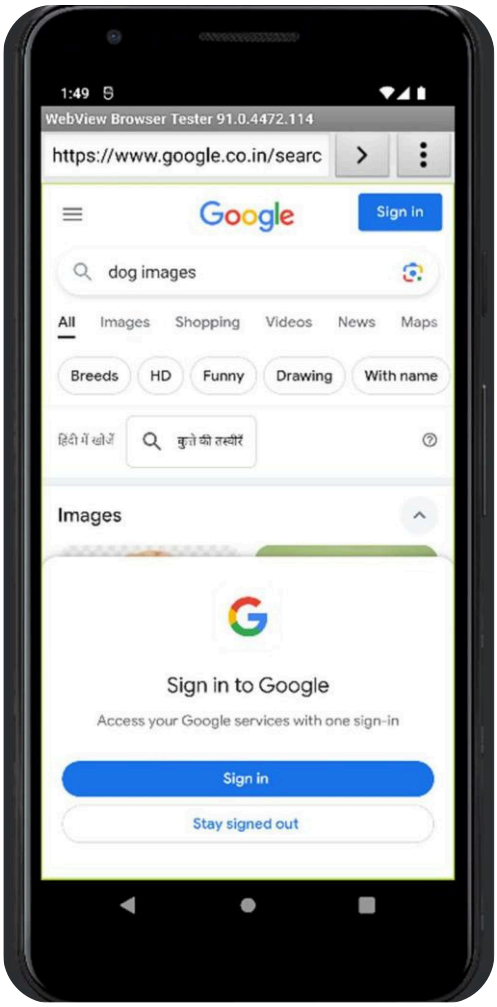
File: activity_main.xml Code:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"    android:layout_height="match_parent"
tools:context=".MainActivity">

   <EditText
      android:id="@+id/search_query_edit_text"
android:layout_width="match_parent"
android:layout_height="wrap_content"       android:hint="Enter search
query"       android:layout_margin="16dp"
android:imeOptions="actionDone"       android:inputType="text"
      android:layout_alignParentTop="true"
android:layout_centerHorizontal="true" />
   <Button
      android:id="@+id/search_button"
android:layout_width="wrap_content"
android:layout_height="wrap_content"       android:text="Search"
      android:layout_below="@id/search_query_edit_text"
android:layout_centerHorizontal="true"
android:layout_marginTop="16dp" /> </RelativeLayout>
```

Output:


1:48
dog images
Search
image's
images
imaged
q w e r t y u i o p
a s d f g h j k l
z x c v b n m
?123


1:49
WebView Browser Tester 91.0.4472.114
https://www.google.co.in/searc
Google
Sign in
dog images
All
Images
Shopping
Videos
News
Maps
Breeds
HD
Funny
Drawing
With name
हिंदी में खोजें
कुत्ते की तस्वीरें
Images
Sign in to Google
Access your Google services with one sign-in
Sign in
Stay signed out

Experiment: 06

Aim: To understand and implement background tasks in Android studio.

Objective:

The primary objective is to learn how to establish network connections send HTTP Requests and handle responses in an Android application, enabling features such as fetching data from API or accessing web services.

Materials:

1. Android studio installed on your computer.
2. Basic understanding of Java programming and Android development.

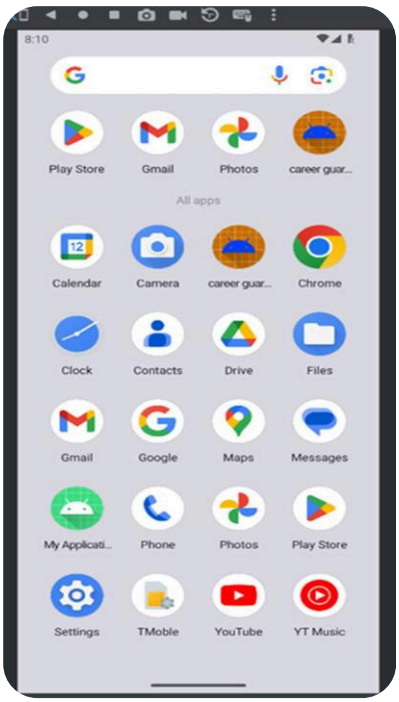
Procedure:

1. Launch Android studio.
2. Create a new project.
3. Implement multiple tasks handling.
4. Display notification popups.
5. Test and debug.

File: activity_main.xml Code:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"  android:layout_width="match_parent"
android:layout_height="match_parent"  android:padding="16dp"  tools:context=".MainActivity">
  <Button  android:id="@+id/button"  android:layout_width="wrap_content"
android:layout_height="wrap_content"  android:text="Send Notification"
android:layout_centerInParent="true"/>
</RelativeLayout>
```

Output:



Experiment: 07

Aim: Overview about SQLite database and it use in android.

Objective:

The objective is to familiarize oneself with the concepts of SQLite databases, including creating, accessing, and manipulating data, and integrating SQLite databases into Android applications.

Materials:

1. Android Studio installed on your computer.
2. Basic understanding of Java programming and Android development.

Procedure:

1. Launch Android studio.
2. Create a new project.
3. Setup the new project.
4. Implement SQLite OpenHelper.
5. Perform Database operations.
6. Test and debug.

SQLite Database:

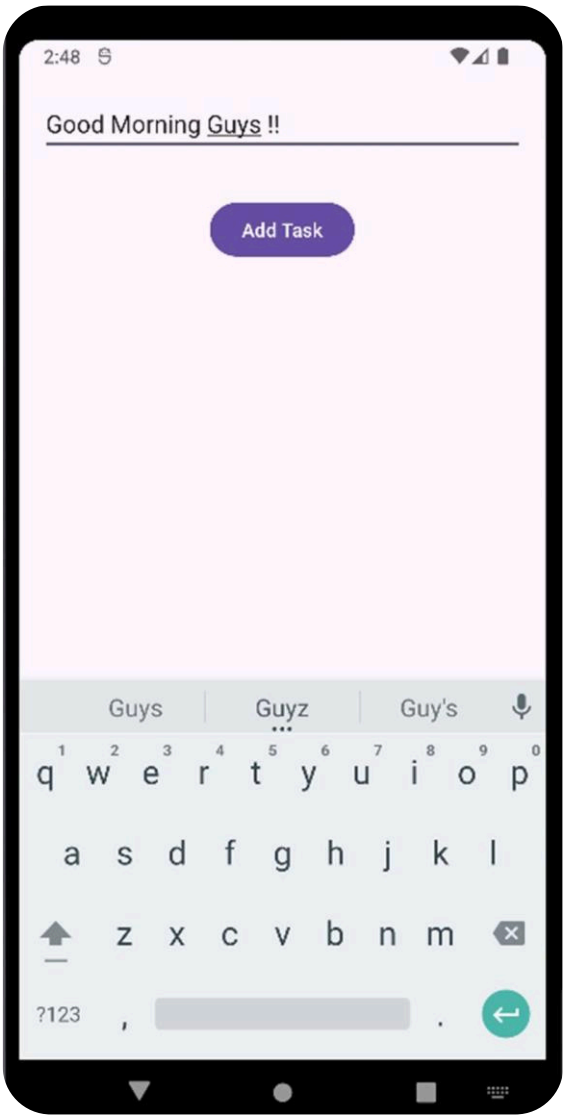
SQLite is a lightweight, embedded, relational database management system (RDBMS) that is widely used in various software applications, including mobile and desktop applications. It is self-contained, serverless, and does not require any configuration or setup, making it easy to integrate into applications.
Implementation: File: activity_main.xml

Code:

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"    android:layout_height="match_parent"
tools:context=".MainActivity">

   <EditText
      android:id="@+id/editTextTask"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter task name"
android:layout_margin="16dp"/>
   <Button
      android:id="@+id/buttonAddTask"
android:layout_width="wrap_content"
android:layout_height="wrap_content"       android:text="Add Task"
      android:layout_below="@id/editTextTask"
android:layout_centerHorizontal="true"
android:layout_marginTop="16dp"/>
</RelativeLayout>
```

Output:



Step 1: Add a new Information. It can be anything and I have added Good Morning Guys!! Step 2: As you can see the Message that “Task Added Successfully”

Experiment: 08

Aim: Storing data using SQLite and sorting data.

Objective:

The objective is to learn how to create and manage SQLite databases in Android applications, store data in them, and implement sorting functionality to display the data in a specific order.

Materials:

1. Android Studio installed on your computer.
2. Basic understanding of Java programming and Android development.

Procedure:

1. Create SQLite Database.
2. Implement Database Helper Class.
3. Insert Data into Database.
4. Implementing Sorting Functionality.
5. Display Sorted Data.

File: activity_main.xml Code:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"     android:layout_height="match_parent"
tools:context=".MainActivity">

    <ListView
android:id="@+id/listView"
android:layout_width="match_parent"
android:layout_height="match_parent" />
</RelativeLayout>
```

Output:

Experiment: 09

Aim: Sharing data and loading data using loader.

Objective:

The objective is to learn how to use Intent to share data between activities and how to implement Loaders to load data asynchronously from content providers or databases.

Materials:

1. Android Studio installed on your computer.
2. Basic understanding of Java programming and Android development.

Procedure:

1. Sharing Data between Activities.
2. Implementing Loaders.
3. Testing and Validation
4. Error Handling and Optimization

File: activity_main.xml Code:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">


   <Button
      android:id="@+id/button"
      android:layout_width="wrap_content"
android:layout_height="wrap_content"       android:text="Send
Data"       android:onClick="sendData"
      android:layout_centerInParent="true"/>

</RelativeLayout>
```

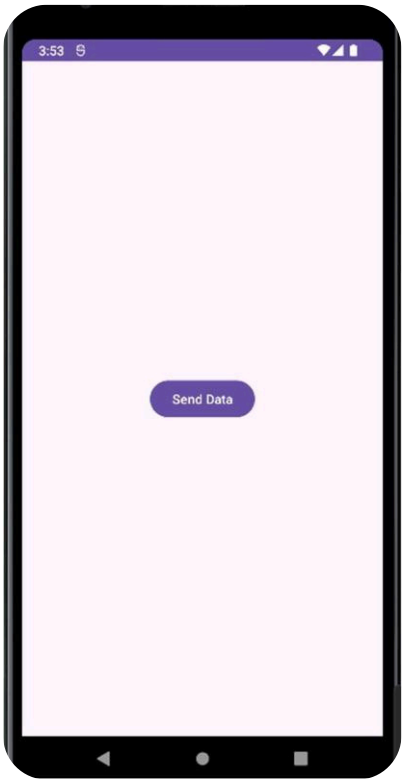
File: activity_second.xml Code:

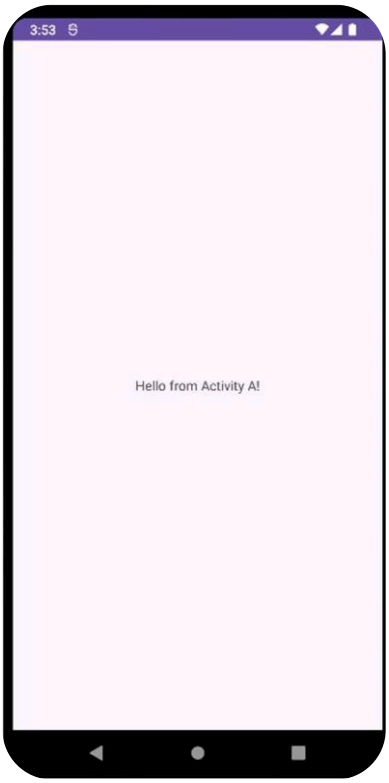
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"     android:layout_height="match_parent"
tools:context=".SecondActivity">

    <TextView
        android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"         android:text="Data will be
displayed here"         android:layout_centerInParent="true"/>

</RelativeLayout>
```

Output:





Experiment 10:

Aim: Testing the app and publish.

Objective:

The objective is to ensure the application functions correctly, is free of bugs, and meets all requirements before publishing it to the Google Play Store.

Materials:

1. Android device or emulator for testing.
2. Google Play Developer account for publishing to the Play Store.
3. Test plan document outlining test cases.

Procedure: 1. Functionality Testing:

- Test all features and functionalities of the application thoroughly.

2. User Interface (UI) Testing:
   - Ensure that the UI elements are displayed correctly and are user-friendly.
3. Performance Testing:
   - Check the app's performance, including loading times, response times, and resource usage.
4. Compatibility Testing:
   - Test the app on di erent Android versions and devices to ensure compatibility.
5. Localization Testing:
   - If the app supports multiple languages, test it with di erent language settings to ensure proper localization.
6. Security Testing:
   - Conduct security testing to identify and fix any vulnerabilities, such as data leaks or insecure network connections.
7. User Acceptance Testing (UAT):
   - Conduct user acceptance testing with a group of testers or beta users to gather feedback on the app's usability and functionality.
8. Publishing:
   - Once all testing is completed and the app is ready for release, publish it to the Google Play Store.
   - Monitor user reviews and feedback after publication and address any issues that arise.
9. Continuous Monitoring and Updates:
   - Monitor the app's performance and user feedback regularly after publication.
   - Release updates as needed to fix bugs, add new features, or improve user experience.

Conclusion:

Thorough testing and preparation are essential steps before publishing an Android application to the Google Play Store. By following a systematic testing process and addressing any issues found, you can ensure that your app provides a smooth and reliable experience for users. Regular updates and maintenance post-publication are also crucial for keeping the app relevant and competitive in the marketplace.